

02.5_data_processing_fundamentals_with_ai_assistance

September 5, 2025

1 Lesson 2.5: Data Processing Fundamentals with AI Assistance

1.1 Bridging AI Communication and Modern Data Tools

Duration: 1 week (Week 4.5)

Weekly Workload: 3-4 hours

Learning Focus: AI-assisted data processing strategies and tool selection for materials science

Course Alignment: This bridge lesson connects prompt engineering skills (Lesson 2) with modern data tools (Lesson 3). Students learn to use AI to guide data processing decisions, tool selection, and workflow optimization in materials science contexts.

1.2 Learning Objectives

By the end of this lesson, you will be able to: - **Use AI to analyze data processing challenges** and recommend appropriate tools - **Apply AI-assisted data cleaning and validation** techniques for materials science datasets - **Design data processing workflows** using AI consultation and iteration - **Compare data processing approaches** with AI-guided performance analysis - **Reflect on AI limitations** in data processing recommendations and validation strategies

1.3 Prerequisites

Complete Lesson 2 - You should have: - Working knowledge of prompt engineering principles - Experience with iterative AI interaction - Understanding of AI limitations and validation needs - Basic Python data manipulation skills from Lesson 1

AI Tools Ready: Claude, ChatGPT, or GitHub Copilot configured and accessible

1.4 Mini-Lecture: AI-Assisted Data Processing Strategy (~20-25 min)

1.4.1 The AI-Augmented Data Processing Workflow

Traditional data processing follows a linear path: **Load** → **Clean** → **Analyze** → **Visualize**. AI augmentation transforms this into an **iterative, consultative process** where AI helps with decision-making at each step.

Core Principles for AI-Assisted Data Processing:

1. **Problem-First Approach** — Start with the materials science question, not the tool

2. **AI Consultation** — Use AI to explore options and trade-offs before committing
3. **Iterative Refinement** — Test AI suggestions, gather feedback, improve approach
4. **Validation-Driven** — Always verify AI recommendations with data and domain knowledge

1.4.2 AI Prompt Patterns for Data Processing

Pattern 1: Data Assessment

ROLE: Data science consultant

AUDIENCE: Materials science researcher

GOAL: Assess [dataset] for processing challenges and tool recommendations

INPUTS: [Dataset description, file size, data types, research question]

CONSTRAINTS: 200 words; include memory/time estimates; suggest 2-3 tool options

TASK: Identify potential issues and recommend processing approach

EVAL CRITERIA: Specific, actionable, considers materials science context

Pattern 2: Workflow Design

ROLE: Data processing specialist

AUDIENCE: Materials engineering team

GOAL: Design optimal workflow for [specific analysis task]

INPUTS: [Data characteristics, performance requirements, constraints]

CONSTRAINTS: 250 words; include step-by-step process; note potential bottlenecks

TASK: Create processing pipeline with tool recommendations

EVAL CRITERIA: Feasible, efficient, materials-science appropriate

Pattern 3: Performance Analysis

ROLE: Performance optimization expert

AUDIENCE: Research computing team

GOAL: Analyze and improve data processing performance

INPUTS: [Current approach, performance metrics, bottlenecks]

CONSTRAINTS: 180 words; suggest specific optimizations; include trade-offs

TASK: Identify bottlenecks and recommend improvements

EVAL CRITERIA: Specific, measurable, considers resource constraints

1.4.3 Materials Science Data Processing Challenges

Common Issues in Materials Datasets: - **Mixed data types:** Composition percentages, temperature ranges, property measurements - **Experimental uncertainty:** Measurement errors, instrument precision, sample variability - **Unit conversions:** SI vs. imperial, different temperature scales, pressure units - **Missing data:** Failed experiments, incomplete measurements, equipment issues - **Large file sizes:** High-resolution microscopy, extensive property databases - **Complex relationships:** Non-linear property correlations, phase transformations

AI Can Help With: - **Tool selection** based on data characteristics and analysis goals - **Workflow optimization** for specific materials science tasks - **Error detection** and data validation strategies - **Performance tuning** for large materials datasets - **Visualization design** for materials property relationships

AI Limitations to Watch For: - **Over-optimization:** May suggest complex solutions for simple problems - **Tool bias:** May favor certain libraries without considering alternatives - **Performance**

estimates: May not account for your specific hardware/constraints - **Domain knowledge gaps:** May miss materials-specific considerations

1.5 Week 4.5 — Grading Rubric (10 pts)

GuidedActivity	Criterion	Excellent (full)	Partial	Pts
1	AI-assisted data assessment	Clear problem framing, specific tool recommendations, materials context	Vague or generic suggestions	4
1	Workflow design & iteration	2+ iterations with KEEP/CHANGE notes, specific improvements	1 iteration or superficial changes	3
2	Performance analysis	Specific bottlenecks identified, measurable improvements suggested	Generic optimization advice	3

Submission: This notebook with prompts, outputs, reflections, and analysis in the Guided Activities below.

Reminder: Each reflection cell tells you the word count requirement and what will be graded. Aim to stay within the word limits, but don't worry if you exceed them slightly.

1.5.1 Guided Activity 1: AI-Assisted Data Processing Strategy (60-75 min)

Your Tasks 1. **Choose a materials science dataset** (use provided sample or your own) 2. **Use AI to assess the dataset** and recommend processing approach 3. **Design a processing workflow** with AI consultation 4. **Iterate on the workflow** based on AI feedback 5. **Reflect on AI recommendations** and validation needs

Materials Science Dataset Options: - **Stress-strain data:** Mechanical properties with experimental uncertainty - **Composition-property data:** Alloy compositions vs. mechanical/thermal properties - **Corrosion data:** Time-series measurements with environmental variables - **Microstructure data:** Image analysis results with particle size distributions - **Thermal analysis data:** DSC/TGA curves with temperature ramps

Tip: Use the prompt patterns above and make sure to include specific materials science context in your prompts.

```
[ ]: # === Dataset Selection ===
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Choose your dataset (uncomment one option or use your own)

# Option 1: Stress-strain data
# data = pd.read_csv('Al7075.csv')
# dataset_type = "Stress-strain data for Al7075 alloy"
# research_question = "Analyze mechanical properties and identify yield_
↳ strength, ultimate strength, and elongation"

# Option 2: Composition-property data (sample)
np.random.seed(42)
data = pd.DataFrame({
    'Al_content': np.random.uniform(85, 99, 100),
    'Cu_content': np.random.uniform(0, 5, 100),
    'Mg_content': np.random.uniform(0, 3, 100),
    'Tensile_Strength_MPa': np.random.uniform(200, 600, 100),
    'Yield_Strength_MPa': np.random.uniform(150, 500, 100),
    'Elongation_Percent': np.random.uniform(2, 25, 100),
    'Hardness_HV': np.random.uniform(50, 200, 100)
})
dataset_type = "Aluminum alloy composition-property database"
research_question = "Identify composition-property relationships and optimize_
↳ alloy design"

# Option 3: Corrosion data (sample)
# time_hours = np.linspace(0, 168, 50) # 1 week
# corrosion_rate = 0.1 * np.exp(-time_hours/48) + 0.02 + 0.01 * np.random.
↳ randn(50)
# data = pd.DataFrame({
#     'Time_hours': time_hours,
#     'Corrosion_Rate_mm_per_year': corrosion_rate,
#     'Temperature_C': 25 + 5 * np.random.randn(50),
#     'pH': 7.5 + 0.5 * np.random.randn(50)
# })
# dataset_type = "Corrosion rate time-series data"
# research_question = "Analyze corrosion kinetics and environmental effects"

print(f"Dataset: {dataset_type}")
print(f"Research Question: {research_question}")
print(f"Dataset shape: {data.shape}")
print(f"\nFirst few rows:")
print(data.head())
print(f"\nData types:")
```

```
print(data.dtypes)
print(f"\nMemory usage: {data.memory_usage(deep=True).sum() / 1024:.1f} KB")
```